

软件安全第十一次实验报告

姓名：林晖鹏 学号：2312966

一、实验名称：

跨站脚本攻击

二、实验要求：

复现课本第十一章实验三，通过img和script两类方式实现跨站脚本攻击，撰写实验报告。

有能力者，可以自己撰写更安全的过滤程序。

三、实验过程：

1. Script方式攻击

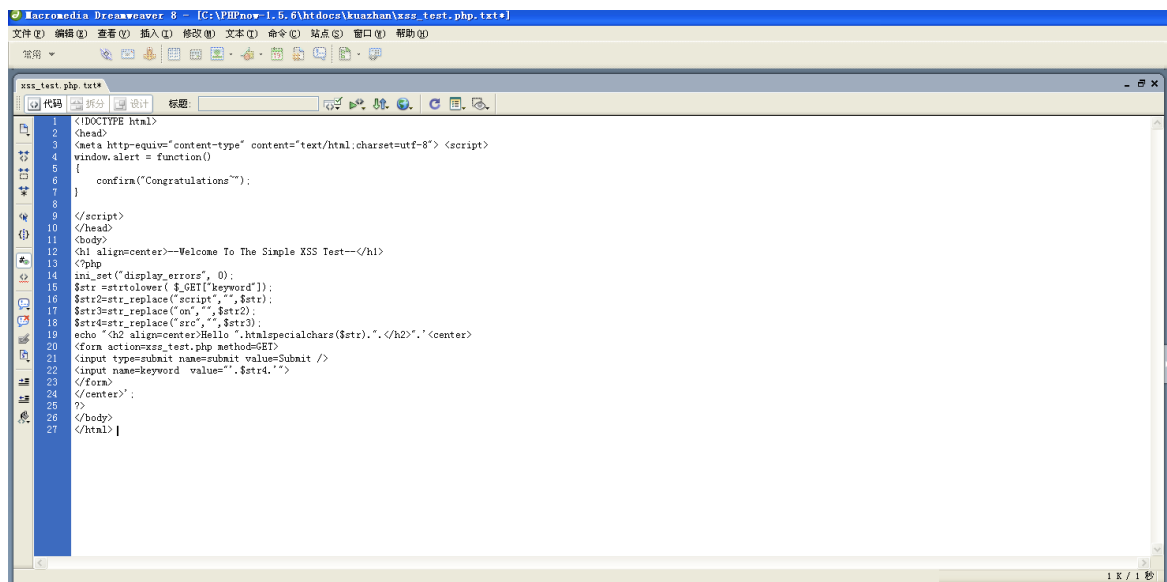
1.1 创建php文件

我们把下面代码写入一个php文件中，类似上次实验一样，放在 `htdocs` 文件夹中。

代码块

```
1  <!DOCTYPE html>
2  <head>
3  <meta http-equiv="content-type" content="text/html; charset=utf-8"> <script>
4  window.alert = function()
5  {
6      confirm("Congratulations~");
7  }
8  </script>
9  </head>
10 <body>
11 <h1 align=center>--Welcome To The Simple XSS Test--</h1>
12 <?php
13 ini_set("display_errors", 0);
14 $str = strtolower( $_GET["keyword"]);
```

```
15 $str2=str_replace("script","",$str);
16 $str3=str_replace("on","",$str2);
17 $str4=str_replace("src","",$str3);
18 echo "<h2 align=center>Hello ".htmlspecialchars($str)."</h2>". '<center>
19 <form action=xss_test.php method=GET>
20 <input type=submit name=submit value=Submit />
21 <input name=keyword value="'. $str4. "'>
22 </form>
23 </center>';
24 ?>
25 </body>
26 </html>
```



1.2 代码分析

我们这里来分析一下上面的这份代码：

- a. 首先是一个script语句，是检测alert函数的发生，一旦发生就生成一个窗口，写着 Congratulations~。

代码块

```
1 <script>
2 window.alert = function()
3 {
4     confirm("Congratulations~");
5 }
6 </script>
```

- b. 然后是一个页面标题显示

代码块

```
1 <h1 align=center>--Welcome To The Simple XSS Test--</h1>
```

- c. 然后是对keyword进行一系列处理，如果是程序刚刚开始，这里的keyword应该为空的。这里进行一系列处理是删除keyword一些部分，防止用户进行渗透攻击。

代码块

```
1 <?php
2 ini_set("display_errors", 0);
3 $str =strtolower( $_GET["keyword"]);
4 $str2=str_replace("script","", $str);
5 $str3=str_replace("on","", $str2);
6 $str4=str_replace("src","", $str3);
7 echo "<h2 align=center>Hello ".htmlspecialchars($str)."</h2>". '<center>
```

- d. 后面是一个输入获取，点击submit按钮就能获取输入，然后经过一系列获取之后，在输入框里面留下处理后的str，标题里面显示处理前的str。

代码块

```
1 <form action=xss_test.php method=GET>
2 <input type=submit name=submit value=Submit />
3 <input name=keyword value="'. $str4. '">
4 </form>
```

1.3 黑盒测试

我们接下来进行黑盒测试，尝试调用alter函数。我们进入网站：

--Welcome To The Simple XSS Test--

Hello .

输入一些安全的字符串，显示的也是输入的字符串。但是包含一些可能会导致alter攻击的字符串，就会被删除。



我们尝试输入一个渗透脚本 `<script>alert('xss')</script>`。发现会被删掉script。

--Welcome To The Simple XSS Test--
Hello `<script>alert('\xss')</script>`.

Submit

但是我们这里可以使用双写来让他删完之后剩下的还是原本语句。

输入 `<scrsriptipt>alert('xss')</scrsriptipt>`。

--Welcome To The Simple XSS Test--
Hello `<scrsriptipt>alert('\xss')</scrsriptipt>`.

Submit

此时，形成了我们想要的脚本语句，但是仍然没有弹出祝贺的弹窗，说明alter函数还是没有被调用。

我们右键网页，点击查看源代码，来观察源代码。

代码块

```
1  <body>
2  <h1 align=center>--Welcome To The Simple XSS Test--</h1>
3  <h2 align=center>Hello &lt;scrsriptipt&gt;alert('\xss')&lt;/scrsriptipt&gt;.
   </h2><center>
4  <form action=xss_test.php method=GET>
5  <input type=submit name=submit value=Submit />
6  <input name=keyword value="<script>alert('\xss')</script>">
7  </form>
8  </center>
```

上面这个是核心部分的网页源代码。我们可以看到，这里被input包起来了，被当成了value，所以没有能当成代码执行，也就没有调用alert函数。

我们希望修改双引号里面的输入，让他绕开过滤条件的同时，又能被当成指令执行。

这里我们在输入中把input结束，让后面部分被识别成指令。

我们输入 `"><script>alert('xss')</script><!--`。这里解释一下。 `">` 是为了把input结束，然后后面的 `<!--` 是因为后面会多出一个input原本的`>`。导致这个`>`直接当成文本显示。这里加的 `<!--` 是为了把后面的`>`隐藏掉。

--Welcome To The Simple XSS Test--

Hello \"><script>alert('xss')</script><!--.

但是依旧没有成功。是因为PHP服务器会自动对输入的双引号进行转义，预防用户构造特殊输入进行攻击。

这里我们在PHPnow安装目录下搜索文件php-apache2handler.ini，并将magic-quotes-gpc=On设置为Off，来关闭这个默认转义。

然后还得去phpnow目录下的pnCmds文件夹下双击一下Apa_Restart这个程序，重启一下php服务器来更新配置。

最后，我们终于看到了我们的congratulations!



1.4 白盒测试

我们重新来观察源代码：

代码块

```
1  <!DOCTYPE html>
2  <head>
3  <meta http-equiv="content-type" content="text/html; charset=utf-8"> <script>
4  window.alert = function()
5  {
6      confirm("Congratulations~");
7  }
8  </script>
9  </head>
10 <body>
11 <h1 align=center>--Welcome To The Simple XSS Test--</h1>
12 <?php
13 ini_set("display_errors", 0);
14 $str = strtolower( $_GET["keyword"]);
15 $str2=str_replace("script","", $str);
16 $str3=str_replace("on","", $str2);
17 $str4=str_replace("src","", $str3);
18 echo "<h2 align=center>Hello ".htmlspecialchars($str). "</h2>". '<center>
19 <form action=xss_test.php method=GET>
20 <input type=submit name=submit value=Submit />
21 <input name=keyword value="'. $str4. "'>
22 </form>
23 </center>';
24 ?>
25 </body>
26 </html>
```

可以看到，这里的过滤条件有三个，分别是删除script、删除on、删除src，这些都是html指令中的一些关键字，我们在黑盒测试里面也没有完全测出这些过滤条件。

这些过滤条件其实相当不完美，导致可以构造更多的恶意输入来进行攻击，下面我们来用ima方式进行攻击。

2. img方式攻击

和上面Script攻击的思路一致，构造合适的输入，让后端执行我们设计的代码。

我们希望构造一个这样的脚本：

代码块

```
1  <img src=ops! onerror="alert('xss')">
```

img标签是用来定义图像的，当src（也就是图片来源）加载错误的时候，就会调用onerror事件，这里就是调用alert函数。

这里我们还是考虑过滤条件差存在，所以对src和on进行双写。所以我们设计输入：

代码块

```
1 "><img ssrsrc=ops! oonerror="alert('xss')"><!--
```



3. 更安全的过滤程序

对于双写的方式，我们可以采用循环删除关键词的方法来保证没有关键词的存在。但是总会有意想不到的攻击输入，如何能把输入正确当成字符串才是最终解法。

纵观上面两个攻击方式，都是因为输入的字符串可以被当成代码写入源代码中。有没有能把输入的字符串不识别成源代码，而是当成代码中的变量的形式，或者说一定能被识别成字符串呢。有的兄弟有的。

`htmlspecialchars()` 是 PHP 中用于防止 XSS 攻击的关键函数之一。它的作用是将 HTML 中的特殊字符转换为安全的实体符号，从而避免浏览器把用户输入解释为 HTML 或 JavaScript 代码。

我们在代码的输出使用这个函数来防止攻击。

代码块

```
1 <!DOCTYPE html>
2 <head>
3 <meta http-equiv="content-type" content="text/html; charset=utf-8"> <script>
4 window.alert = function()
5 {
6     confirm("Congratulations~");
```

```
7 }
8 </script>
9 </head>
10 <body>
11 <h1 align=center>--Welcome To The Simple XSS Test--</h1>
12 <?php
13 ini_set("display_errors", 0);
14 $str =strtolower( $_GET["keyword"]);
15 $str2=str_replace("script","", $str);
16 $str3=str_replace("on","", $str2);
17 $str4=str_replace("src","", $str3);
18 echo "<h2 align=center>Hello ".htmlspecialchars($str)."</h2>".<center>
19 <form action=xss_test.php method=GET>
20 <input type=submit name=submit value=Submit />
21 <input name=keyword value="'.htmlspecialchars($str4).'">
22 </form>
23 </center>';
24 ?>
25 </body>
26 </html>
```

此时，前面两个攻击都失效了:(

四、实验总结

本次实验中，我们通过Script和img两种方式进行跨站脚本攻击，这里都是反射式XXS。归根到底，还是向后端代码中注入恶意代码的思路，让输入的一部分成为源代码中的一部分，从而让程序执行我设计的代码。

然后我们用了 `htmlspecialchars` 函数来设计更加安全的程序，此时过不过滤已经无所谓了，输入已经无法被识别成代码了。

但是值得思考的是，这个 `htmlspecialchars` 函数总归有一个具体实现代码，在这个函数里面的实现代码进行XXS攻击，是不是也能绕过这个函数呢？
